

GPT-2

Causal pretraining

GPT-2 turns causal next-token prediction into one text interface for generation and text-specified tasks.

Language Models are Unsupervised Multitask Learners

Alec Radford^{*1} Jeffrey Wu^{*1} Rewon Child¹ David Luan¹ Dario Amodei^{**1} Ilya Sutskever^{**1}

Radford et al. (2019)

One objective supports many text-defined tasks

GPT-2 combines causal next-token modeling, text-serialized tasks, naturally occurring demonstrations, and zero-shot evaluation without target-task parameter or architecture changes.

- **Causal objective** Predict each next token using only the prefix already available.
- **Task as sequence** Place task identity, input, and desired output in one symbol stream.
- **Natural demonstrations** Web text contains examples, conventions, and input-output patterns without explicit task curation.
- **Zero-shot interface** Change the evaluation text rather than the target-task weights or architecture.

These are design claims. They do not establish that every task is learned, correct, or reliable.

Radford et al. (2019), §§1–2

Causal masking turns representations into generation

BERT reconstructs

BERT can use tokens on both sides when reconstructing a selected position.

river [MASK] eroded

left context ← bank → right context

The output is a contextual representation of a corrupted input, not an efficient left-to-right generation loop.

Devlin et al. (2019), §§1–3 · Radford et al. (2019), §2

GPT-2 continues

GPT-2 restricts each prediction to the prefix already available.

river → bank → eroded

prefix → next token → longer prefix

- Append the selected token, expose a longer prefix, and repeat.
- The same prediction operation becomes a generation loop as the prefix grows.

Next-token likelihood creates a repeatable generation loop

Using index-consistent notation for GPT-2 §2, Eq. (1):

$$p(x) = \prod_{i=1}^n p(s_i \mid s_1, \dots, s_{i-1})$$

- **Factor** i scores token s_i using only the prefix $s_1, \dots, s_{i-1}$.
- **Training** changes weights to raise likelihood across many text sequences.
- **Generation** keeps weights fixed while the prefix grows one token at a time.

Likelihood specifies the generation mechanics; it does not guarantee that the model interprets the intended task.

Task, input, and output become one sequence

Goal. Serialize the task, input, and expected output into one causal continuation.

translate to French · the book is blue · le livre est bleu

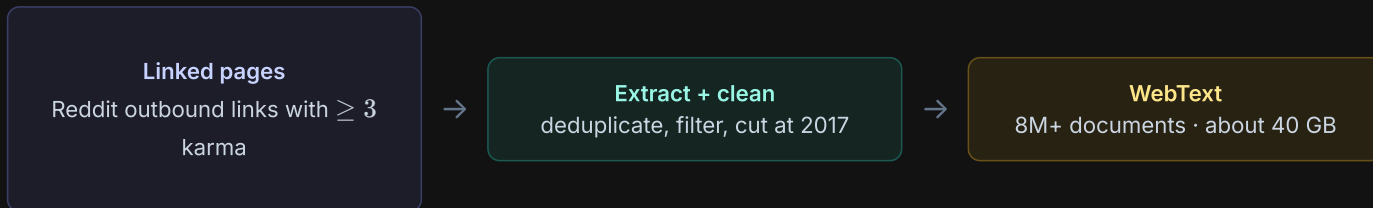
translate to French · the book is blue · ...

- **Training sequence:** task instruction, English input, then the French continuation.
- **Inference prefix:** provide the task and English input, then stop before the output.
- **The model** continues from that prefix with unchanged weights.

A prompt changes context, not parameters—and does not guarantee that the continuation is correct.

Course-authored example instantiating the task sequence in Radford et al. (2019), §2.

WebText supplies demonstrations without task labels



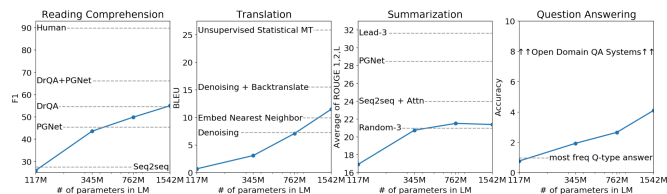
- Table 1 places English and French phrases beside translations inside ordinary passages rather than labeled task rows.
- WebText begins from outbound Reddit links receiving at least three karma, a human-filtering heuristic rather than a neutral sample.
- Text extraction, deduplication, heuristic cleaning, a 2017 cutoff, and Wikipedia removal further shape the training distribution.
- The preliminary corpus contains slightly over eight million documents and about 40 GB of text.

Radford et al. (2019), §2 and Table 1

Larger GPT-2 models improve across four zero-shot probes

Across the four task-family panels in Figure 1, reported zero-shot behavior generally strengthens with model size.

- Read each panel along the common model-size progression from 117M to 1542M parameters.
- The four panels cover reading comprehension, translation, summarization, and question answering.
- The direction generally improves with size, but panels use task-specific metrics and reference lines.
- A cross-task trend does not establish uniform competence or comparable absolute gains.



Radford et al. (2019), Figure 1, p.2

Task-as-text remains an uneven, data-dependent interface

GPT-2 does not establish uniform task reliability, robust instruction following, contamination-free evaluation, factuality, alignment, or grounded generation.

Zero-shot outcomes vary across task families and evaluation conventions.

WebText selection and overlap make the training distribution part of every transfer claim.

The report states that its largest model still underfits WebText as train and test perplexity fall with scale.

GPT-3 therefore asks whether richer context and greater scale strengthen fixed-weight adaptation.

How far can a fixed-weight model adapt from context alone?

Radford et al. (2019), Figure 1 and §4, pp.2, 8–9

GPT-3

In-context task learning

GPT-3 turns the prompt from a task label into a fixed-weight task-learning interface.

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan [†]	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	

Descriptions and demonstrations change the prediction problem

GPT-3 makes task learning at evaluation a controlled change to context rather than a parameter update.

- **No gradient update** Evaluation changes the prefix, not the model parameters.
- **Three prompt regimes** Zero-, one-, and few-shot settings isolate how much task information appears in context.
- **Scale as intervention** A family up to 175B parameters tests whether context use strengthens with model size.
- **Broad but bounded evaluation** Many tasks are aggregated while task variation and contamination remain explicit limitations.

The intervention is visible: change context or scale, then observe the fixed-weight prediction.

Brown et al. (2020), pp.1, 3–7

Zero-, one-, and few-shot change context—not weights

Zero-shot

instruction $\rightarrow$ query $\rightarrow$?

Zero-shot: task description plus query.

One-shot

example $\rightarrow$ query $\rightarrow$?

One-shot: one input-output demonstration precedes the query.

Few-shot

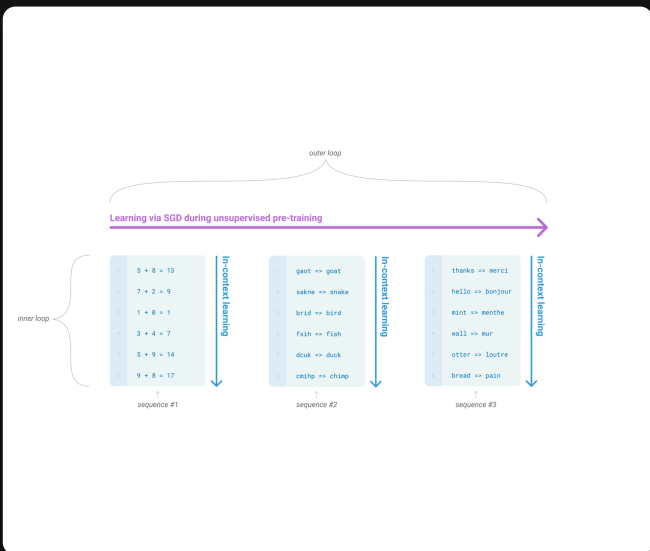
examples $\times K \rightarrow$ query $\rightarrow$?

Few-shot: several demonstrations precede the query.

Only the context changes; evaluation parameters remain fixed.

Brown et al. (2020), Figure 2.1, p.7

Pretraining changes weights; demonstrations change predictions



- Outer process: gradient updates accumulate across pretraining documents.
- Inner context: demonstrations and the query occupy one evaluation sequence.
- The model predicts an answer conditionally on that sequence with fixed parameters.
- The paper's inner-learning analogy is a framing, not proof of internal gradient descent.

Two timescales; only the outer one changes weights.

A few examples define one classification continuation

Review: "Sharp, warm, and beautifully paced." Label: positive

Review: "Long, flat, and never convincing." Label: negative

Review: "Uneven, but the final act lands." Label: ?

1. Demonstration 1 pairs a short review with the token `positive`.
2. Demonstration 2 pairs another review with the token `negative`.
3. Append a new review followed by an empty label slot.
4. Compare the next-token probabilities of the permitted label strings without updating weights.

Course-authored trace implementing Brown et al. (2020), Figure 2.1

A weighted corpus mixture shapes the fixed model

- Filtered Common Crawl supplies most raw tokens.
- WebText2, Books1, Books2, and Wikipedia are oversampled relative to raw size.
- Corpus size, sampled training tokens, and epochs describe different quantities.
- The resulting pretraining distribution is curated rather than a neutral population.

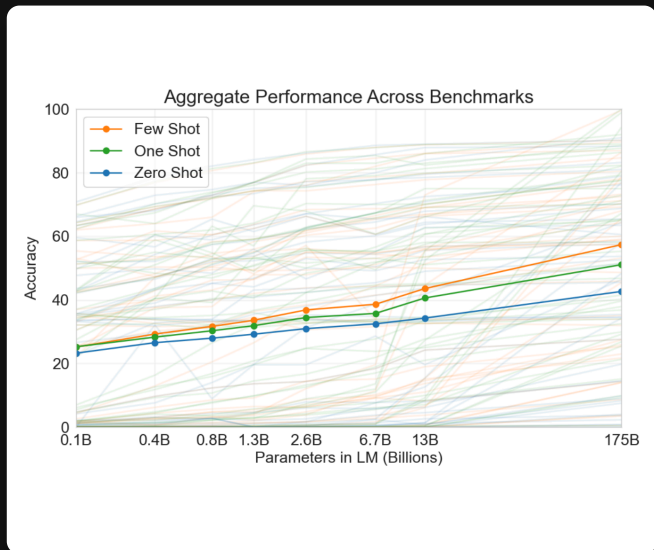
Sampling weight is a design decision that enters every later prompt response.

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

Brown et al. (2020), Table 2.2, p.9

Scale strengthens the aggregate in-context interface



Brown et al. (2020), Figure 1.3, p.5

Figure 1.3 reports aggregate performance generally increasing with model size and additional in-context demonstrations.

- Zero-, one-, and few-shot aggregate curves generally rise with model size.
- Few-shot is usually highest in the reported aggregate.
- Many faint trajectories reveal heterogeneous task behavior behind the average.
- One accuracy-denominated aggregate does not establish any individual task's reliability.

Read the colored aggregate and faint task trajectories together.

In-context learning is an interface, not a guarantee

Aggregate gains do not establish uniform task reliability, prompt robustness, efficient context use, contamination-free evaluation, or alignment.

Individual tasks and example counts vary behind the aggregate.

Demonstrations consume context and inference compute.

Section 4 flags possible benchmark contamination and reports filtered comparisons.

When context alone is insufficient, the next design question is how much of the model must change.

When context is insufficient, must adaptation update every weight?

Brown et al. (2020), Figure 2.1 and §4

LoRA

Low-rank adaptation

LoRA makes task adaptation a small versioned update rather than a full checkpoint copy.

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation

Hu et al. (2021)

A full checkpoint is not the only adaptation surface

LoRA changes the parameterization of adaptation while preserving the pretrained base path.

- **Frozen base** The pretrained matrix remains unchanged during task adaptation.
- **Low-rank delta** Two trainable factors produce the additive update BA .
- **Targeted placement** Adaptation acts on selected Transformer matrices rather than every parameter.
- **Mergeable serving** The learned delta can stay versioned separately or be merged into the base weight for one served task.

The base computation stays; adaptation becomes an explicit additive object.

Hu et al. (2021), Figure 1, §4.1, §7.1

Frozen and trainable paths share the same input

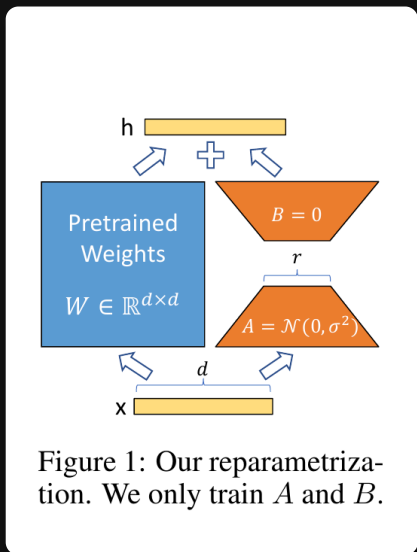


Figure 1: Our reparametrization. We only train A and B .

Hu et al. (2021), Figure 1, p.1

- W_0x is the frozen pretrained path and receives no task gradient.
- A compresses the input into r learned directions.
- B expands those directions back to the layer's output dimension.
- The two paths meet through the additive output $W_0x + BAx$.

$$x \rightarrow W_0x$$

+

$$x \rightarrow A \rightarrow B \rightarrow BAx$$

The learned update lives in a rank- r **subspace**

$$\Delta W = BA$$

$$A \in \mathbb{R}^{r \times k}, \quad B \in \mathbb{R}^{d \times r}, \quad r \ll \min(d, k)$$

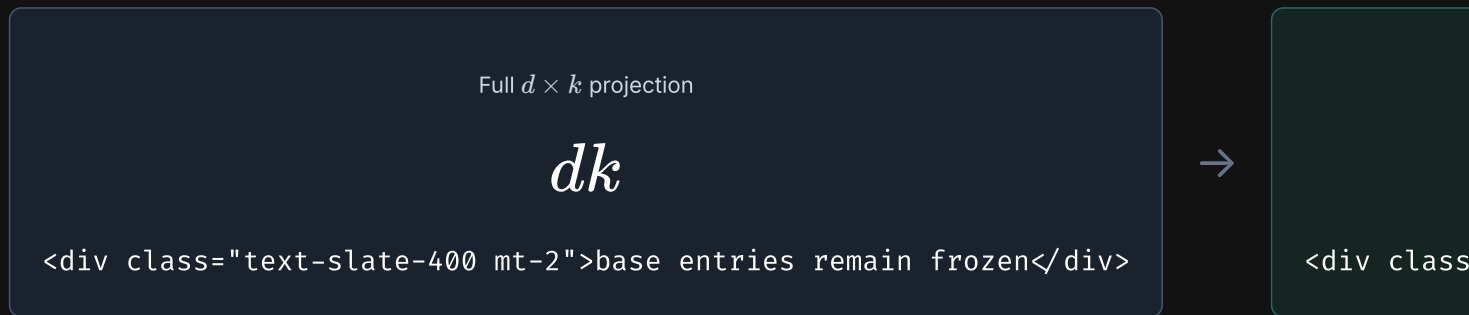
$$h = W_0 x + \Delta W x = W_0 x + BAx$$

scale $\Delta W x$ by α/r ; for serving compute $W = W_0 + BA$

- A has shape $r \times k$ and B has shape $d \times r$, with r much smaller than d and k .
- The product BA is the only learned weight update and has rank at most r .

The forward pass adds the scaled update to

Rank sets capacity; placement sets where change can act



1. Start with one frozen $d \times k$ projection containing dk base entries.
2. Choose rank r , then train $r(d + k)$ entries across A and B while the base remains fixed.
3. Attach the delta to selected projections; the paper's central GPT-3 setup targets query and value matrices.
4. Store BA as a versioned adapter or merge $W_0 + BA$ when serving one task.

Small trainable updates remain competitive on three reported tasks

Table 4 reports competitive WikiSQL, MNLI, and SAMSum results with substantially fewer trainable parameters than full fine-tuning in the studied configurations.

- Read the trainable-parameter column beside each task metric.
- Compare LoRA with full fine-tuning and the listed adapter baselines.
- The bounded result spans WikiSQL, MNLI, and SAMSum in the reported setup.
- Competitive rows do not establish universal equivalence across tasks or models.

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	RI/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^{rl})	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^{rl})	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

Hu et al. (2021), Table 4, p.8

Parameter efficiency does not select desirable behavior

Low-rank parameter efficiency does not select objectives or data and does not guarantee behavior quality, serving isolation, robustness, privacy, safety, or alignment.

Rank and placement remain task- and model-dependent choices.

A compact delta can inherit or introduce undesirable behavior.

Separate adapters still require host-owned versioning, isolation, and evaluation.

The next question is what supervision and objective should shape outputs.

How can post-training shape broad instruction-following behavior?

Hu et al. (2021), §4.1, Table 4, §7.1

InstructGPT

Human-feedback post-training

InstructGPT makes post-training a data-and-objective pipeline for selecting behavior, not merely a larger pretraining run.

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell[†] Peter Welinder Paul Christiano^{*†}

Jan Leike* Ryan Lowe*

OpenAI

Three learned artifacts turn feedback into policy changes

InstructGPT turns different forms of human feedback into distinct datasets and learned artifacts.

- **Supervised policy** Labeler demonstrations teach an initial response policy.
- **Preference model** Ranked completions train a scalar comparison model.
- **Constrained policy update** PPO increases learned reward while penalizing drift from SFT and optionally retaining pretraining pressure.
- **Distribution-bound evaluation** Human preference is measured on the studied API-prompt distribution and evaluator protocol.

Feedback changes form at every stage—and each form trains a different artifact.

Ouyang et al. (2022), Figures 1–2, §§3.2–3.5

Three stages transform three forms of supervision

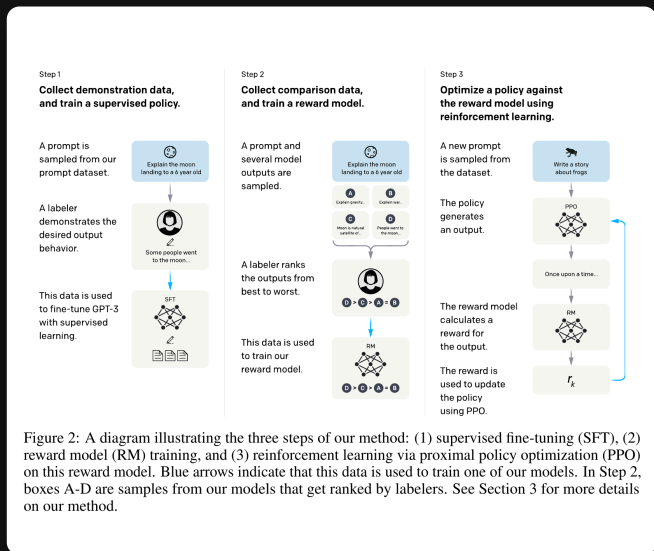
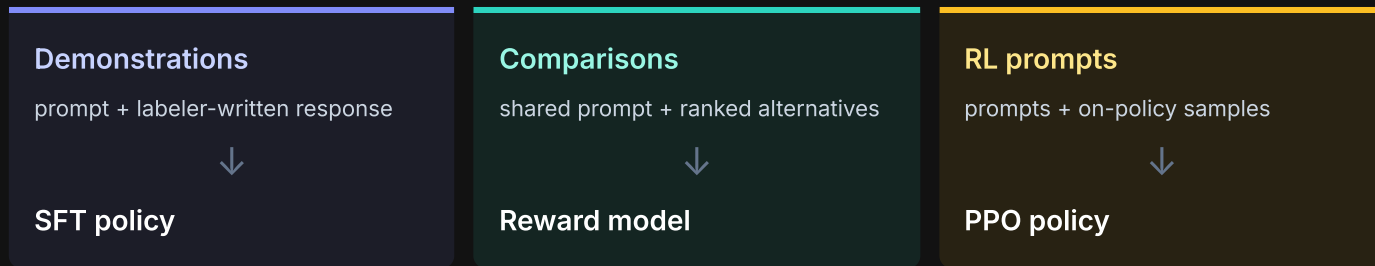


Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

1. Stage 1: prompt-response demonstrations produce the SFT policy.
2. Stage 2: ranked completions produce the reward model.
3. Stage 3: newly sampled completions and learned rewards update the policy with PPO.
4. Each arrow changes both the supervision object and the learned artifact.

Read each arrow as: data object → training rule → learned model.

Each stage needs a different feedback object



- SFT data contains labeler-written prompt-response demonstrations.
- PPO data contains prompts used to sample and score new policy responses.
- Reward-model data contains multiple completions and pairwise rankings for the same prompt.
- Prompt sources, train-validation splits, and reuse rules differ across stages.

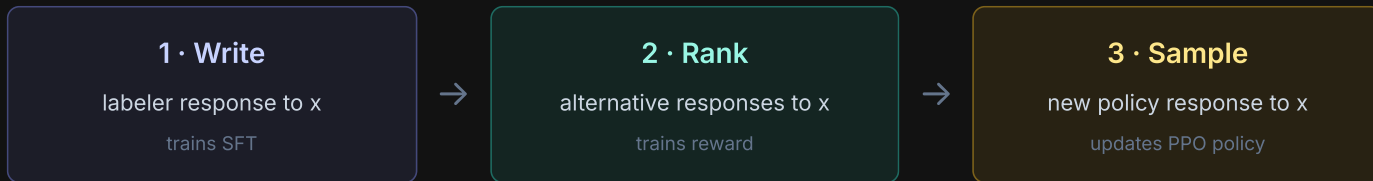
Ouyang et al. (2022), §§3.2–3.4 and Table 6

Pairwise rankings train a relative reward

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

- One prompt x is shared by the preferred completion y_w and rejected completion y_l .
- The reward model outputs one scalar $r_\theta(x, y)$ for each completion.
- Training raises $r_\theta(x, y_w)$ relative to $r_\theta(x, y_l)$ through a log-sigmoid loss.
- The learned scalar represents sampled comparative preference, not a truth label.

One prompt passes through demonstration, comparison, and policy update



1. Demonstration: a labeler writes a desired response to prompt x for supervised fine-tuning.
2. Comparison: several SFT-policy responses to x are ranked and converted into reward gaps.
3. Policy sample: the current policy generates a new response whose learned reward drives PPO.
4. Constraint terms penalize drift from the SFT reference through β and retain pretraining pressure through γ for PPO-ptx.

Course-authored trace from Ouyang et al. (2022), Figure 2 and Eqs. (1)–(2)

Labelers prefer the reported post-trained policies

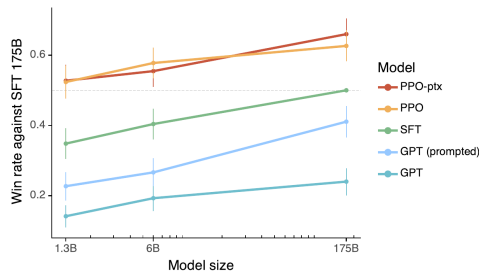


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

Ouyang et al. (2022), Figure 1, p.2

Figure 1 reports post-trained policies winning labeler comparisons against the 175B SFT baseline on the studied API prompts.

- The y-axis is labeler win rate against the 175B SFT baseline.
- Compare GPT, prompted GPT, SFT, PPO, and PPO-ptx conditions across model sizes.
- Post-trained policies lead in the reported evaluation, including the smaller 1.3B policy.
- Preference is specific to the prompt distribution, comparison protocol, and evaluators.

The result is preference under this protocol—not direct truth or safety.

Human feedback selects behavior within a sampled world

Preference optimization remains bounded by selected labelers, sampled prompts and candidates, labeling instructions, evaluator agreement, and reward-model generalization.

Labeler selection, demographics, training, and instructions shape the rankings.

The API-prompt distribution defines the evaluated use cases.

Candidate responses limit what the comparison process can prefer.

Chain-of-Thought next asks whether intermediate generated tokens can improve a fixed-weight answer before any new reward training.

Can intermediate generated tokens improve a difficult answer without updating weights?

Ouyang et al. (2022), §§3.1–3.4, Figure 3, §5

Chain-of-Thought puts intermediate steps into the continuation

Chain-of-Thought turns generated tokens before the answer into a test-time resource for multistep problems without changing model weights.

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou
Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Wei et al., NeurIPS 2022

A worked rationale changes what the model must generate

Chain-of-Thought changes the demonstration and completion program while the model weights remain fixed.

- **Rationale-bearing exemplars** Each demonstration contains intermediate steps before its answer rather than only an input-output pair.
- **Rationale-before-answer decoding** The test completion emits a variable-length intermediate sequence before committing to the answer.
- **One schema across tasks** Arithmetic, commonsense, planning, and symbolic problems use the same input-to-rationale-to-answer ordering.
- **Ablation and scale tests** The paper separates coherent steps from extra tokens and reports where the effect appears across model scale.

Wei et al. (2022), Figures 1 and 3–6, physical pp.1, 4–6.

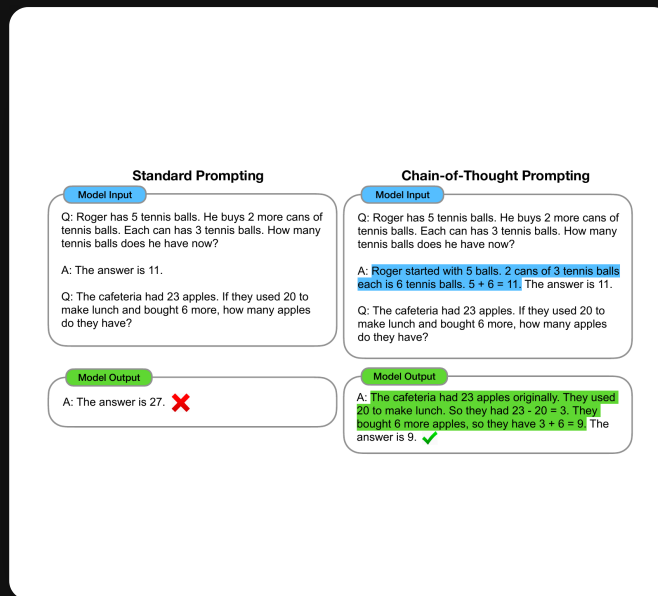
The answer moves behind a generated rationale

Direct answer: input → answer.

Chain-of-Thought: input → rationale tokens → answer.

The rationale can name subproblems, intermediate quantities, or a short plan.

At evaluation, only the context and generated continuation change; the model weights remain fixed.



Wei et al. (2022), Figure 1, physical p.1.

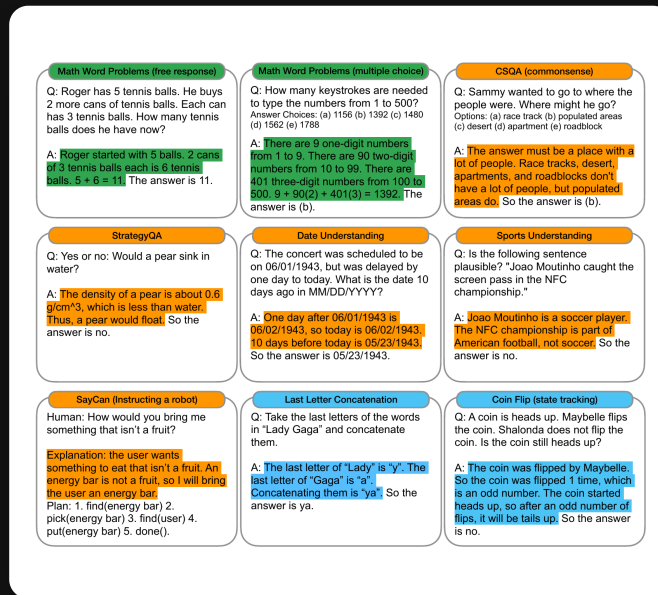
One rationale schema carries different kinds of intermediate work

Arithmetic — compute and carry intermediate quantities.

Commonsense — connect stated facts to an answer choice.

Planning — translate an instruction into ordered robot actions.

Symbolic tasks — transform structured inputs step by step while preserving the same input → rationale → answer order.



Wei et al. (2022), Figure 3, physical p.4.

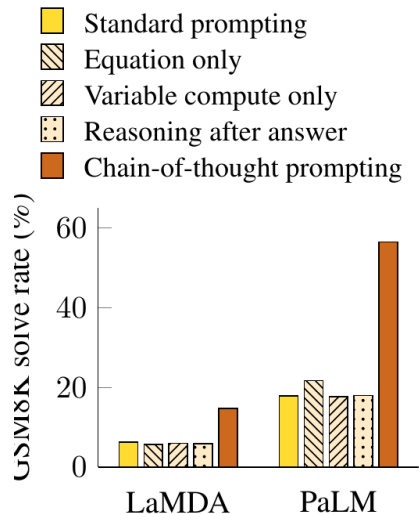
Three controls test what the rationale contributes

Equation only — output a mathematical equation before the answer but omit natural-language decomposition.

Variable compute only — emit a dot sequence equal to the number of characters in the equation needed to solve the problem, adding tokens without coherent intermediate steps.

Reasoning after answer — generate the rationale only after the answer has already been produced.

Full Chain-of-Thought — place coherent intermediate language before the answer.



Wei et al. (2022), Section 3.3 and Figure 5, physical pp.5–6.

The reported separation appears mainly at the largest scales

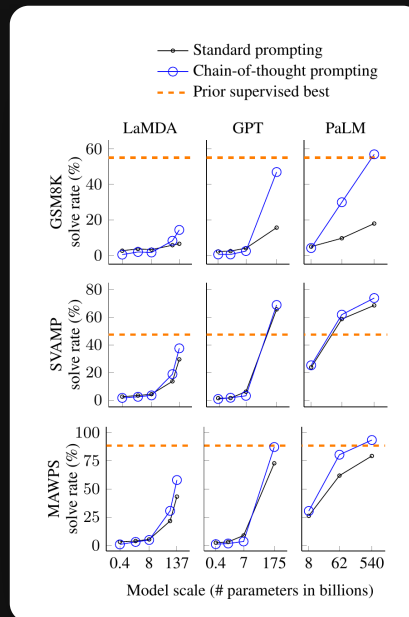
On the three arithmetic benchmarks plotted in Figure 4, the reported Chain-of-Thought benefit is scale-dependent and is largest at the largest tested model scales.

Read each benchmark panel separately: GSM8K, SVAMP, and MAWPS.

Compare standard and Chain-of-Thought prompting within the same model family and scale.

Smaller models do not receive a uniform benefit.

The strongest reported gaps occur at the largest tested scales, not at a universal threshold.



Wei et al. (2022), Figure 4 and Section 3.2, physical p.5.

A useful rationale remains prompt-sensitive and unverified

Prompt sensitivity and improved task accuracy leave rationale causality and faithfulness unverified.

- Different annotators produce different rationale wording and measured solve rates.
- A concise style changes the generated reasoning surface.
- Independent exemplar sets change the context on which the fixed model conditions.
- None of these tests shows that the visible rationale is a faithful causal trace.

Can post-training make reasoning behavior emerge without labeled rationales?

Wei et al. (2022), Section 3.4 and Figure 6, physical p.6.

DeepSeek-R1 turns checkable outcomes into a reasoning signal

DeepSeek-R1 moves reasoning from a prompt-only behavior to a post-training target driven by checkable final outcomes.

Article

DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning

Daya Guo, Dejian Yang, Haowei Zhang et al. · complete 194-contributor registry on pp.5–6 and in provenance
Guo, Yang, Zhang et al., Nature 645 (2025), DOI 10.1038/s41586-025-09422-z

Outcome rewards, relative updates, and staged data shape the policy

The pipeline labels final outcomes where they are checkable, compares sampled responses within a group, and adds staged supervised and preference data where a rule verifier is insufficient.

- **Rule-based outcomes** Accuracy and format checks score mathematics, coding, and logical-reasoning responses.
- **Group-relative optimization** GRPO estimates each sampled response's advantage from rewards within its group instead of training a separate value model.
- **Cold-start reasoning data** A small supervised seed establishes readable long-form reasoning before the first reasoning-focused RL stage.
- **Broadening and final RL** Rejection-sampled reasoning data, broader SFT data, and later rule-based plus preference rewards produce the final policy.

Guo, Yang, Zhang et al. (2025), Figures 1–2; Methods, physical p.7; Extended Data Fig. 2, p.10.

GRPO uses the response group as the baseline

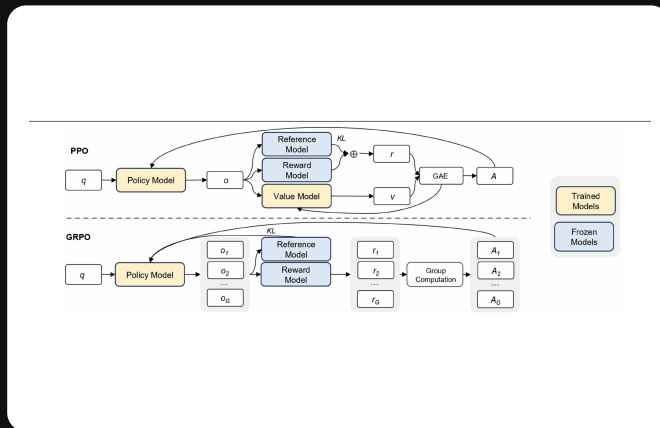
$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

Sample a group $\{o_1, \dots, o_G\}$ from the old policy for the same question q .

Score every response and compute A_i by centering and scaling r_i with the group's reward mean and standard deviation.

Increase probability for relatively better responses under the clipped policy objective.

Penalize divergence from a reference policy; removing the value critic does not remove the RL objective or KL constraint.



Guo, Yang, Zhang et al. (2025), Methods Eq. 3, p.7; Extended Data Fig. 2, p.10.

The rule reward checks the endpoint and its required format

$$\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}}$$

Accuracy reward — exact mathematical answers or compiler tests check final correctness where a dependable rule exists.

Format reward — enclosing the reasoning process within the designated `<think>` and `</think>` tags receives a separate rule-based score.

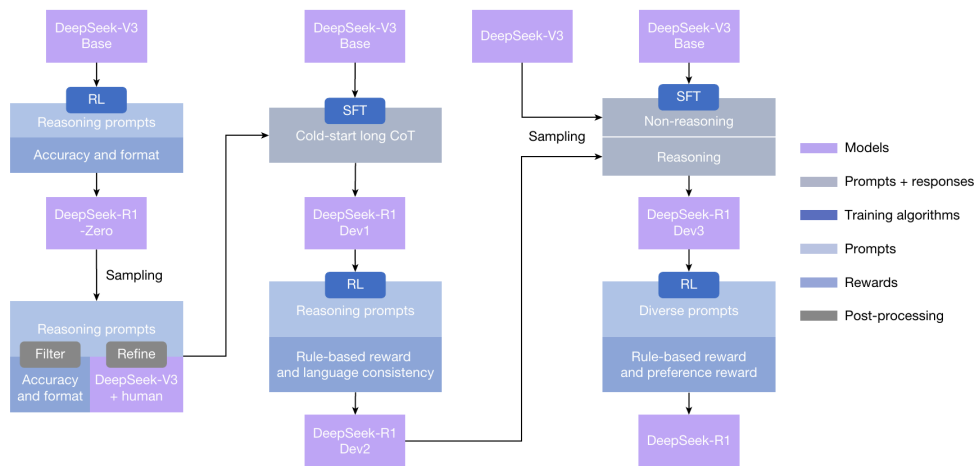
required markers: `<think> ... </think>`

The paper combines them as $\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}}$.

A correct endpoint can coexist with an invalid, inefficient, or unfaithful intermediate path.

Guo, Yang, Zhang et al. (2025), Methods “Reward design,” Eq. 4, physical p.7.

The final policy is built in four distinct stages



Cold-start SFT — seed a readable long-rationale format before large-scale reasoning RL.

Rejection sampling and SFT — filter or refine generated reasoning data and combine it with broader supervised data.

Guo, Yang, Zhang et al. (2025), complete Figure 2 stage trace, p.3; Methods, pp.7–8.

Reasoning RL — optimize sampled responses on reasoning-oriented prompts with rule-based rewards.

Final RL — train on diverse prompts with rule-based reasoning rewards and model-based preference rewards.

One sampled solution can become both reward evidence and training data

prompt → response group → rule scores → policy update or retained data

1

Sample — generate a complete rationale and final answer for one verifiable prompt.

2

Score — apply accuracy and format rules, then compare the response with others sampled for the same prompt.

3

Update — use the group-relative advantage in the reasoning-focused RL stage.

4

Reuse and broaden — retain an eligible generated response for supervised data, combine with broader data, and continue to the final mixed-reward RL stage.

Source-bound course trace from Guo, Yang, Zhang et al. (2025), Fig. 2, p.3; Methods, p.7; Extended Data Fig. 2, p.10.

AIME accuracy rises across one reported R1-Zero training run

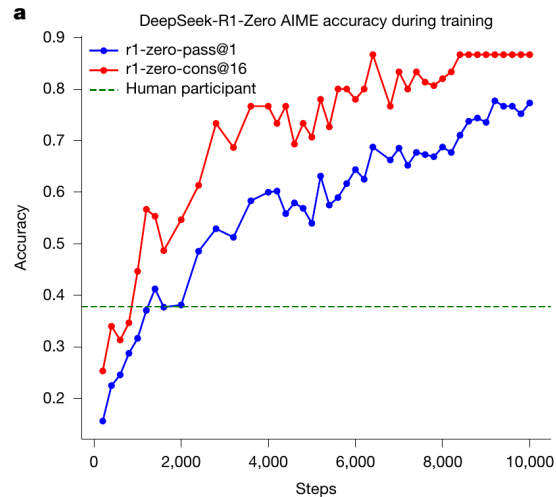
For the paper's reported DeepSeek-R1-Zero run on AIME 2024, pass@1 rises across training and consensus@16 remains higher; this is one benchmark trajectory, not general reasoning competence.

Read training steps on the horizontal axis and AIME accuracy on the vertical axis.

Track pass@1 separately from consensus@16 rather than collapsing them into one score.

Both reported series rise across the plotted run.

The figure does not isolate which pipeline component caused the change.



Guo, Yang, Zhang et al. (2025), Figure 1a, physical p.2.

Reliable outcome rewards cover only part of reasoning

Verifiable-task success does not establish general, efficient, safe, robust, or faithful reasoning where dependable outcome checks are unavailable.

- Verifier coverage — dependable rule rewards are difficult for open-ended tasks such as writing.
- Process gap — a correct endpoint does not validate each intermediate claim or establish trace faithfulness.
- Deployment limits — the paper records overthinking, language mixing, prompt sensitivity, weak structured output, and absent tool use.
- Optimization risk — model-based rewards can be exploited as training progresses.

How can a model consult knowledge outside its parameters when its internal reasoning is insufficient?

Guo, Yang, Zhang et al. (2025), limitations, physical pp.4–5; reward design, p.7.

RAG places an external document index in the generation path

RAG moves part of the system's knowledge from fixed parameters into an explicit external store that can be searched and replaced.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{1,2}, Ethan Perez²,

Aleksandra Piktus³, Fabio Petroni³, Vladimir Karpukhin³, Naman Goyal³, Heinrich Küttler³,

Mike Lewis³, Wen-tau Yih³, Tim Rocktäschel^{1,3}, Sebastian Riedel^{1,3}, Douwe Kiela³

¹Facebook AI Research; ²University College London; ³New York University;
plewis@fb.com

Lewis et al., NeurIPS 2020

Retrieval makes the document a latent input to generation

RAG combines parametric generation with explicit non-parametric memory and trains the query encoder and generator through a shared latent-document objective.

- **Dense retrieval** A query encoder scores passages stored in a precomputed document index.
- **Document-conditioned generation** The generator reads the query and each retrieved passage together.
- **Latent-document marginalization** Retriever probabilities weight alternative document-conditioned generations.
- **Two factorizations** RAG-Sequence shares one latent document across an answer, while RAG-Token can reweight documents at each token.

Lewis et al. (2020), Figure 1 and Sections 2.1–2.3, physical pp.2–3.

Query, retrieve, condition, then marginalize

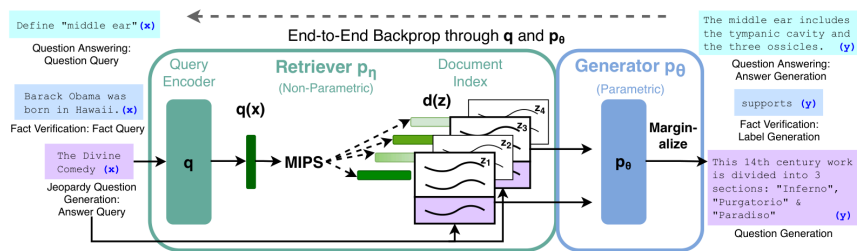


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder + Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

Encode query x and compare it with indexed document representations.

Retrieve top- k passages z and retain their weights $p_\eta(z \mid x)$.

Concatenate x with each z and compute the generator distribution $p_\theta(y_i \mid x, z, y_{1:i-1})$.

Marginalize document-conditioned predictions rather than treating one retrieved passage as automatically correct.

RAG-Sequence sums one document choice over the whole answer

$$\begin{aligned} p_{\text{RAG-Sequence}}(y \mid x) &\approx \sum_{z \in \text{top-}k(p(\cdot \mid x))} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z) \\ &= \sum_{z \in \text{top-}k(p(\cdot \mid x))} p_{\eta}(z \mid x) \prod_i^N p_{\theta}(y_i \mid x, z, y_{1:i-1}) \end{aligned}$$

- The retriever supplies top- k latent documents z and weights $p_{\eta}(z \mid x)$.
- For one z , the generator scores the complete output as a product over token probabilities.
- The outer sum marginalizes the one-document choice across the complete sequence.
- The resulting probability still does not certify that any passage supports the answer.

RAG-Token can change its document mixture at every token

One document term per answer

RAG-Sequence — choose one latent document term and score the whole answer under it.

A new mixture at each position

RAG-Token — recompute the weighted document mixture for each output position.

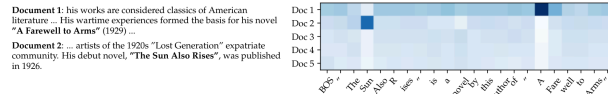


Figure 2: RAG-Token document posterior $p(z_i | x, y_i, y_{-i})$ for each generated token for input "Hemingway" for Jeopardy generation with 5 retrieved documents. The posterior for document 1 is high when generating "A Farewell to Arms" and for document 2 when generating "The Sun Also Rises".

In Figure 2, posterior mass shifts toward different retrieved documents as different book-title tokens are generated.

Token-level flexibility does not turn posterior mass into a citation or verified attribution.

Lewis et al. (2020), Section 2.1, p.3; Figure 2, p.7.

Retrieval improves several reported open-domain QA scores

On the paper's listed open-domain QA test sets, RAG variants improve several exact-match results relative to the shown baselines, but no one RAG factorization dominates every task.

Read Natural Questions, TriviaQA, WebQuestions, and CuratedTREC as separate task columns.

Compare RAG-Token and RAG-Sequence with the listed closed-book and open-book baselines.

The stronger RAG variant differs by task rather than winning uniformly.

Exact-match accuracy does not establish document relevance, citation, or claim support.

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Lewis et al. (2020), complete Table 1 and footnotes, physical p.6.

Retrieved text is available evidence, not verified support

External memory makes passages available but does not establish retrieval relevance, source quality, currentness, attribution, faithful use, or claim-level support.

- Retrieval failure — the needed passage is absent from top-k.
- Source failure — an available passage is stale, low-quality, conflicting, or adversarial.
- Use failure — the generator ignores or misreads relevant retrieved content.
- Support failure — the final claim outruns what any retrieved passage establishes.

Can reasoning and action alternate as observations arrive?

Course synthesis grounded in Lewis et al. (2020), physical pp.1–3 and Table 1, p.6.

ReAct

Reasoning meets environmental feedback

ReAct turns the action-observation loop into an explicit part of the model's test-time trajectory.

REACT: SYNERGIZING REASONING AND ACTING IN
LANGUAGE MODELS

Shunyu Yao^{*1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

Yao et al. (ICLR 2023)

Observations can redirect the next step

ReAct lets external observations revise the next reasoning step instead of committing to a complete plan before interaction begins.

- **Interleaved trajectory** Free-form thoughts and task-specific actions alternate in one context.
- **Observation feedback** Environment responses become new inputs rather than post-hoc evidence.
- **Plan revision** Later reasoning can redirect the trajectory when a lookup fails or state changes.
- **One loop across tasks** The same reason-act-observe pattern supports knowledge lookup and interactive environments.

The loop makes new evidence actionable; it does not make the evidence trustworthy.

Yao et al. (2023), §§1–2

Thought → Action → Observation creates a feedback loop

Thought — identify the unresolved fact, subgoal, or exception.

→ **Action** — choose one operation from the task's allowed interface.

→ **Observation** — append the environment's response to the trajectory.



Revise — condition the next thought or action on the expanded context.

The environment creates the new observation; the model only chooses the proposed action.

Yao et al. (2023), Figure 1d and §2

Language actions extend context; environment actions change state

$$o_t \in \mathcal{O}; \quad a_t \in \mathcal{A}; \quad \pi(a_t \mid c_t)$$

$$c_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$$

$$\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}; \quad \hat{a}_t \in \mathcal{L}$$

$$c_{t+1} = (c_t, \hat{a}_t)$$

Base policy

The base policy chooses an environment action a_t from $\mathcal{A}$ using trajectory context c_t .

Language action

A language action $\hat{a}_t$ extends context to $c_{t+1} = (c_t, \hat{a}_t)$ without producing an environment observation.

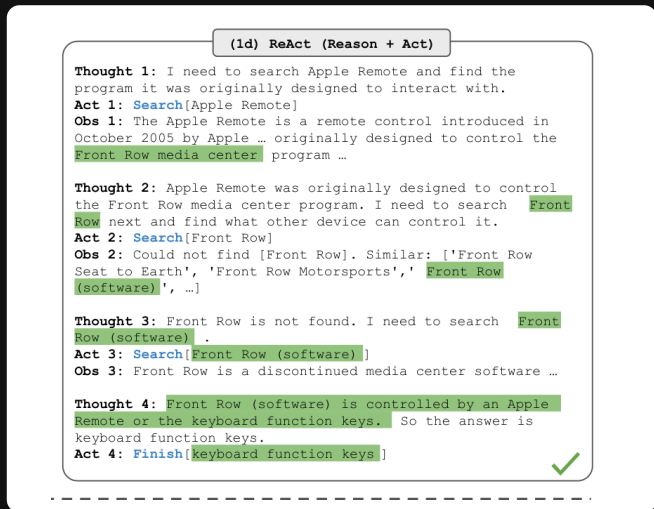
Augmented space

The augmented space $\mathcal{A} = \mathcal{A} \cup \mathcal{L}$ adds free-form language actions.

Environment action

An environment action changes or queries the task environment and returns the next observation.

A missing fact triggers search, observation, and replanning



Yao et al. (2023), Figure 1d

Trace the revision points

1. Identify the missing relation between Apple Remote and the program it controlled.
2. Issue `search[Apple Remote]` and read the observation naming Front Row.
3. Reformulate the failed Front Row lookup as `search[Front Row software]`.
4. Use the new observation to finish with keyboard function keys.

One successful trace cannot certify the retrieved observations.

The host turns action text into a WebShop transition

Action: think[For 3 ounce bottle of bright citrus deodorant for sensitive skin, the item has options 'bright citrus' and '3 ounce (pack of 1)' and seems good to buy.]
Observation: OK.

Action: click[bright citrus]
Observation: You have clicked bright citrus.

Action: click[3 ounce (pack of 1)]
Observation: You have clicked 3 ounce (pack of 1).

Action: click[Buy Now]

Constrain

The benchmark constrains actions to search, choose products or options, and buy.

Execute

The host parses and executes the command against the sandbox website.

Emit

The model emits think, search, or click text with arguments.

Observe

The returned page or status becomes the next observation; Buy Now terminates the benchmark trajectory.

Interleaving helps differently across knowledge tasks

Prompt Method ^a	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	64.6
ReAct → CoT-SC	35.1	62.0
Supervised SoTA^b	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

F

Table 1 reports that plain ReAct outperforms Act on HotpotQA and FEVER, beats Chain-of-Thought on FEVER, trails it on HotpotQA, and that hybrid prompting rows perform best among the listed prompting methods.

- Compare ReAct with Act to isolate the value of generated reasoning around actions.
- ReAct improves FEVER over Chain-of-Thought but does not improve HotpotQA over it.
- The strongest prompting row differs by task between the two hybrid orderings.
- The result is specific to the listed model, prompts, datasets, and metrics.

A trajectory is not an authorization policy

ReAct defines an adaptive trajectory but does not supply a deployment protocol for authorization, credentials, confirmation, rollback, monitoring, or control.

- A coherent command can still request an unsafe or out-of-scope operation.
- Observations can carry errors or adversarial instructions into the next context.
- The host must own permissions, credentials, confirmation, execution, and audit state.
- The next foundation needs an architecture that locates model, memory, actions, and environment explicitly.

What architecture separates the model from memory, actions, and the environment?

Course synthesis grounded in Yao et al. (2023), Figure 1, §2, Table 6

CoALA

A cognitive architecture around the model

CoALA turns prompts, memories, and tools into an explicit system architecture with separable state and operations.

Cognitive Architectures for Language Agents

Theodore R. Sumers* Shunyu Yao* Karthik Narasimhan Thomas L. Griffiths
Princeton University
{sumers, shunyuy, karthikn, tomg}@princeton.edu

Sumers et al. (TMLR 2024)

CoALA names memory, action, and decision boundaries

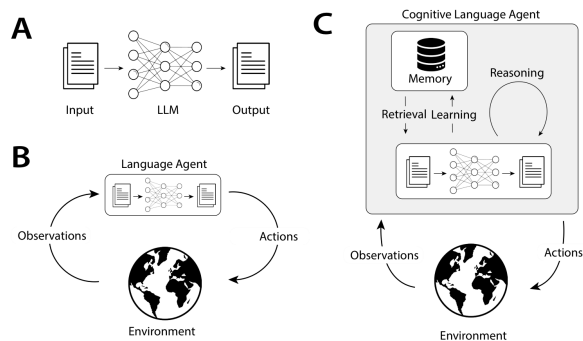
CoALA separates the probabilistic model from host-managed state, operations, and control flow.

- **Memory taxonomy** Working, episodic, semantic, and procedural memory play different roles.
- **Structured action space** Reasoning, retrieval, learning, and external grounding are distinct operations.
- **Decision cycle** Observation, planning, selection, execution, and feedback repeat over time.
- **Comparative vocabulary** Different language-agent designs can be reconstructed using the same component language.

The taxonomy tells us where to ask trust questions; it does not answer them.

Sumers et al. (2024), §§1, 4

An environment closes the loop around the model



Sumers et al. (2024), Figure 1

- **Language model** — text input produces a distribution over text output.
- **Language agent** — host code converts observations into text and output text into actions.
- **Environment** — executing an action produces the next observation.
- **Cognitive agent** — memory, reasoning, retrieval, and learning also modify internal state between calls.

Prompting methods become controlled production sequences

Prompting Method	Production Sequence
Zero-shot	$Q \overset{\text{LLM}}{\rightsquigarrow} Q A$
Few-shot	$Q \longrightarrow Q_1 A_1 Q_2 A_2 Q \overset{\text{LLM}}{\rightsquigarrow} Q_1 A_1 Q_2 A_2 Q A$
Retrieval Augmented Generation	$Q \xrightarrow{\text{Wiki}} Q O \overset{\text{LLM}}{\rightsquigarrow} Q O A$
Socratic Models	$Q \overset{\text{VLM}}{\rightsquigarrow} Q O \overset{\text{LLM}}{\rightsquigarrow} Q O A$
Self-Critique	$Q \overset{\text{LLM}}{\rightsquigarrow} Q A \overset{\text{LLM}}{\rightsquigarrow} Q A C \overset{\text{LLM}}{\rightsquigarrow} Q A C A$

Table 1: Conceptual diagram illustrating how prompting methods manipulate the input string before generating completions. Q = question, A = answer, O = observation, C = critique, and $\rightsquigarrow$ denotes sampling from a stochastic production. These pre-processing manipulations – which can employ other models such as vision-language models (VLMs), or even the LLM itself – can be seen as productions. Prompting methods thus define a *sequence* of productions.

A model call samples one continuation from the distribution conditioned on its input string.

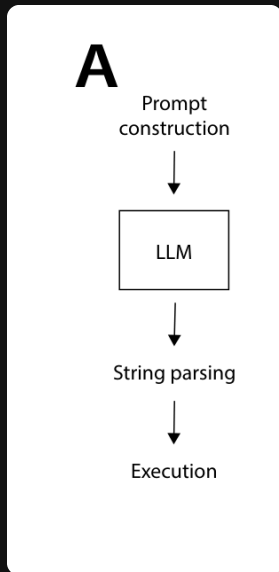
Few-shot prompting prepends demonstrations before the sampled completion.

Retrieval-augmented prompting inserts an external observation before the model answers.

Self-critique routes an initial answer through critique and refinement calls.

Sumers et al. (2024), §§3.1–3.2, Table 1

The host turns one model call into an operation



Sumers et al. (2024), Figure 3A

1. **Construct**

Prompt construction selects a template and inserts variables from working memory.

2. **Generate**

The LLM returns a text continuation rather than executing an operation itself.

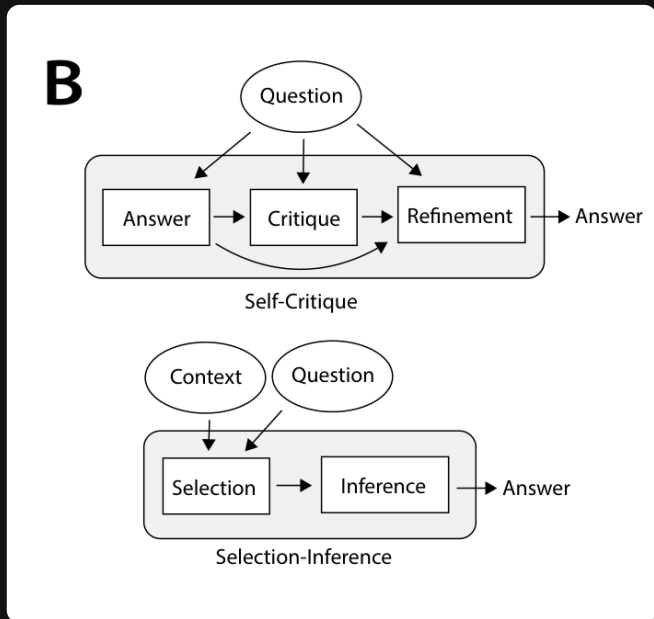
3. **Parse**

String parsing maps output text into an answer, function call, or action form.

4. **Execute**

Host execution applies the parsed operation and owns any resulting state change.

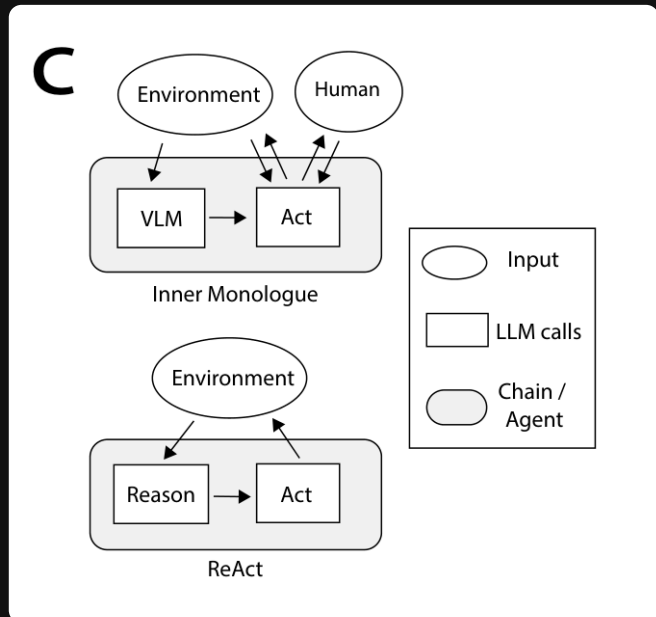
A prompt chain is a fixed host program



Sumers et al. (2024), Figure 3B

- Self-Critique follows answer → critique → refinement → answer in a host-defined order.
- Selection-Inference follows context → selection → inference → answer in a host-defined order.
- Intermediate model outputs become variables for the next scheduled call.
- The chain does not become adaptive merely because it contains multiple calls.

Feedback makes the sequence adaptive

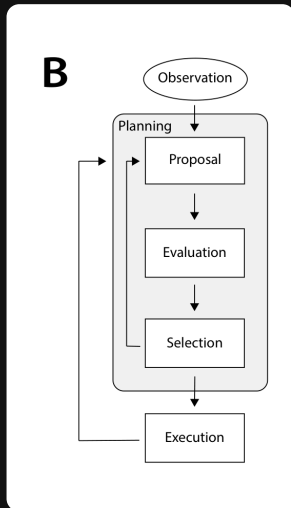


1. Convert the current environment observation into model-readable state.
2. Use the model and host procedure to select a reason or act step.
3. Parse and execute the selected external action through the host interface.
4. Feed the resulting observation back so the next cycle can change course.

Adaptive to observations does not mean observations are trustworthy or actions are authorized.



A decision cycle still needs independent trust checks



Sumers et al. (2024), Figure 4B and §4.6

A component taxonomy and decision cycle do not establish grounding, authorization, robustness, privacy, interpretability, evaluation validity, monitoring, or control.

1. **Observe** — load current environment and working-memory state.
2. **Plan** — use reasoning and retrieval to propose and evaluate candidate operations.
3. **Select and execute** — apply one grounding or learning action through agent code.
4. **Verify next** — later modules test source support, permissions, robustness, privacy, monitoring, and control at the exposed boundaries.

When the full composition produces an answer or action, how should later modules test whether it is grounded, safe, and controllable?